System for Artificial Intelligence

Introduction to ML Systems

Siyuan Feng
Shanghai Innovation Institute

OUTLINE

01 Intro to Deep Learning

02 ML Systems Overview

01

Intro to Deep Learning

Machine Learning



Object recognition

Cat

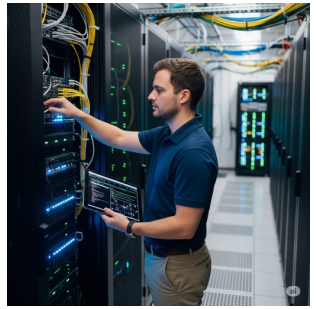
The capital of China is

Text generation (next-token prediction)

Beijing

An AI Infra engineer

Image generation



Machine Learning



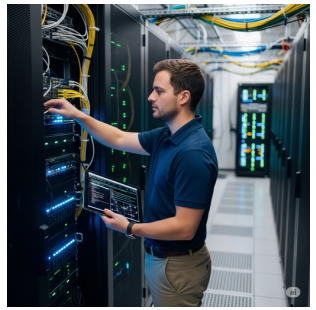
The capital of China is

An AI Infra engineer

$$x \xrightarrow{f_\theta(x)} y$$

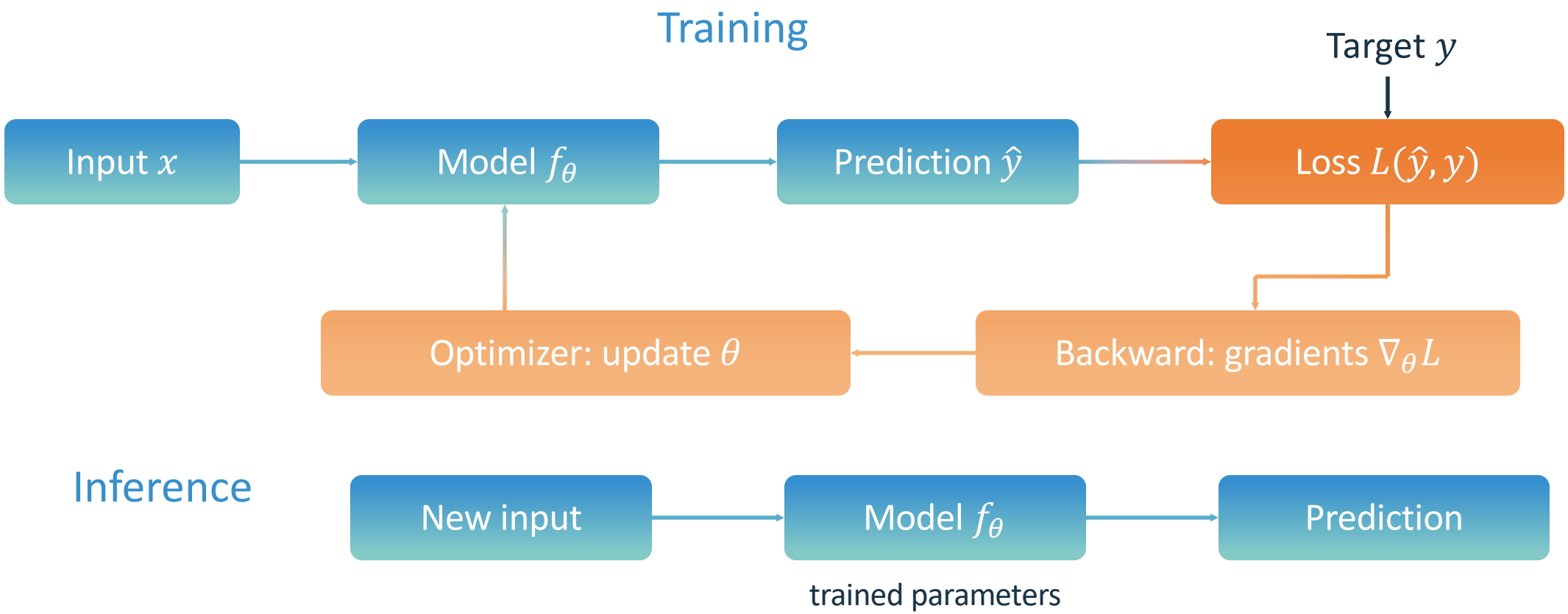
Cat

Beijing



Machine learning learns a parameterized function from data.

Training and Inference



Training changes model parameters. Inference uses them.

One Parameter Update

$$\hat{y} = wx \quad L = \frac{1}{2}(\hat{y} - y)^2 \quad x = 2 \quad y = 6 \quad \eta = 0.1$$

1. Forward → 2. Backward → 3. Update

$$w = 1$$
$$\hat{y} = 2$$
$$L = 8$$

$$\frac{\mathrm{d}L}{\mathrm{d}\hat{y}} = -4$$
$$\frac{\mathrm{d}\hat{y}}{\mathrm{d}w} = 2$$
$$\frac{\mathrm{d}L}{\mathrm{d}w} = -8$$

$$w = 1 - 0.1 \times (-8)$$
$$= 1.8$$
$$\text{new } L = 2.88$$

The backward pass supplies a gradient. The optimizer applies the update.

Tensors and Operators

上海创智学院
Shanghai Innovation Institute

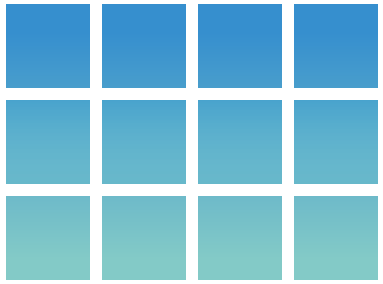
X: input



$[\text{batch} = 2, \text{features} = 3]$

×

W: parameters



$[3,4]$

=

Y: output



$[\text{batch} = 2, \text{outputs} = 4]$

Tensor: shape + dtype + device

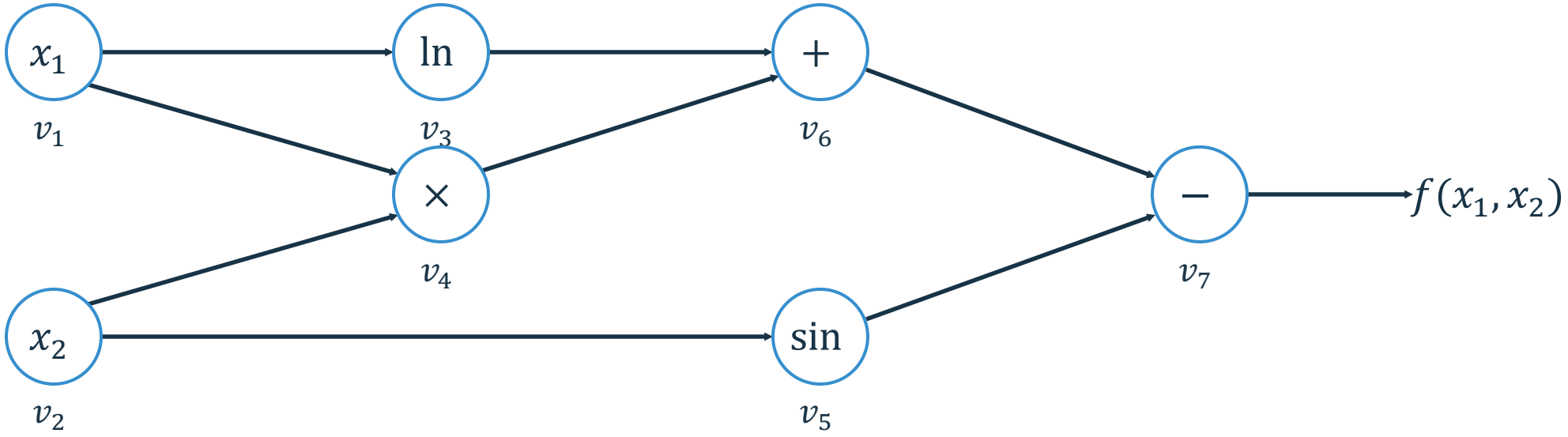
Operator: a computation on tensors

Computational Graph Abstraction

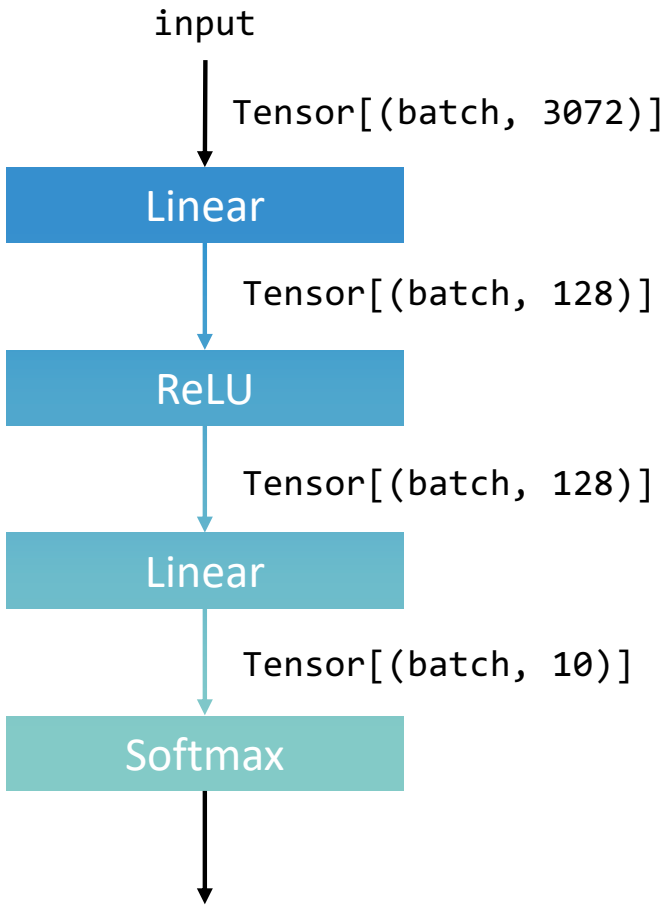
上海创智学院
Shanghai Innovation Institute

- Nodes represent inputs or operations.
- Edges carry values and encode data dependencies.

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$



Computational Graph for a Classification Model



Tensor is the central data format in deep learning models

Deep Learning Architectures and MoE

上海创智学院
Shanghai Innovation Institute

- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers
- Mixture-of-Experts (conditional computation)

MoE layers can be used within Transformer architectures.

CNNs are Widely Used in Vision Tasks



Object Detection

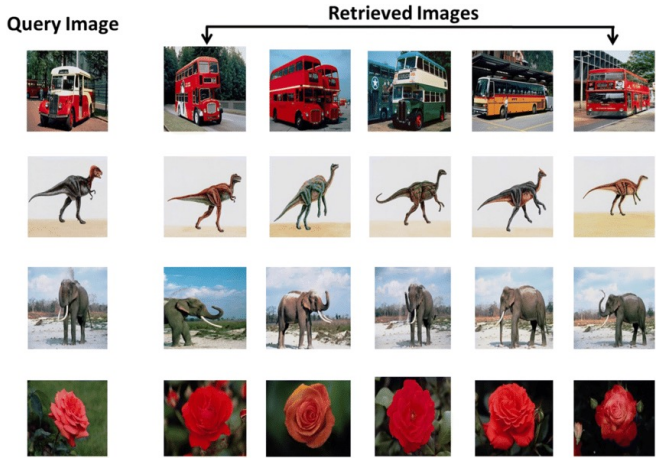


Image Retrieval



Self-Driving

Convolution

上海创智学院
Shanghai Innovation Institute

Convolve the filter with the image: slide over the image spatially and compute dot products

Input image

0	0	0	0	0
0	0	0	0	0
0	0	1	1	0
0	0	1	2	0
0	0	0	0	0

3×3 neighborhood

×

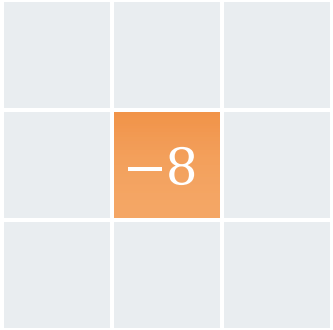
Kernel

4	0	0
0	0	0
0	0	−4

Elementwise products, then sum

=

Output feature map

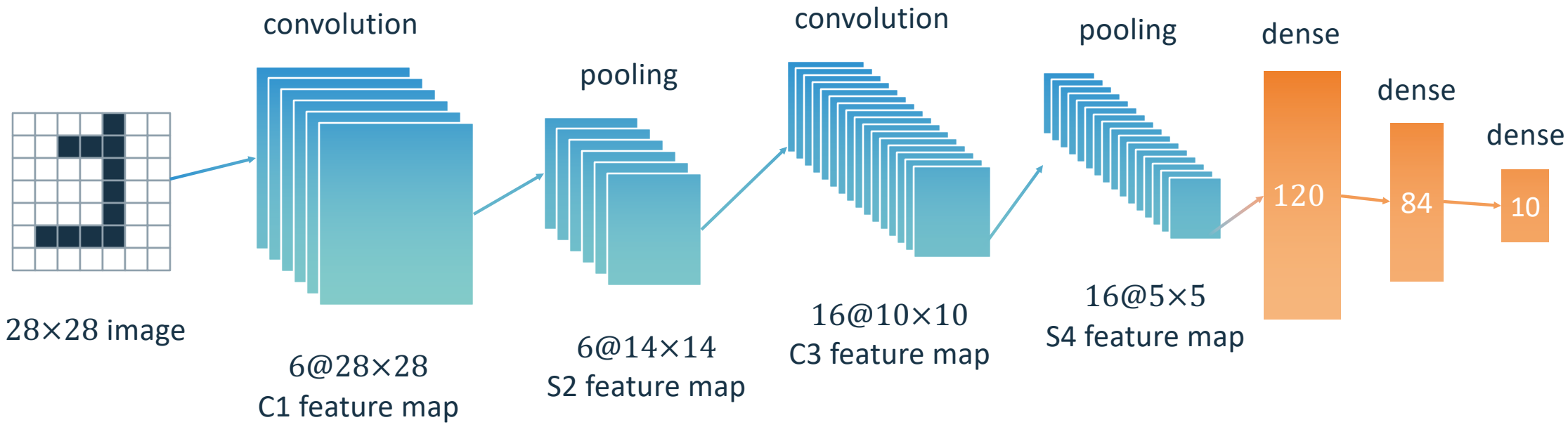


$$4\times0 + (-4)\times2 = -8$$

Convolutional Neural Networks

上海创智学院
Shanghai Innovation Institute

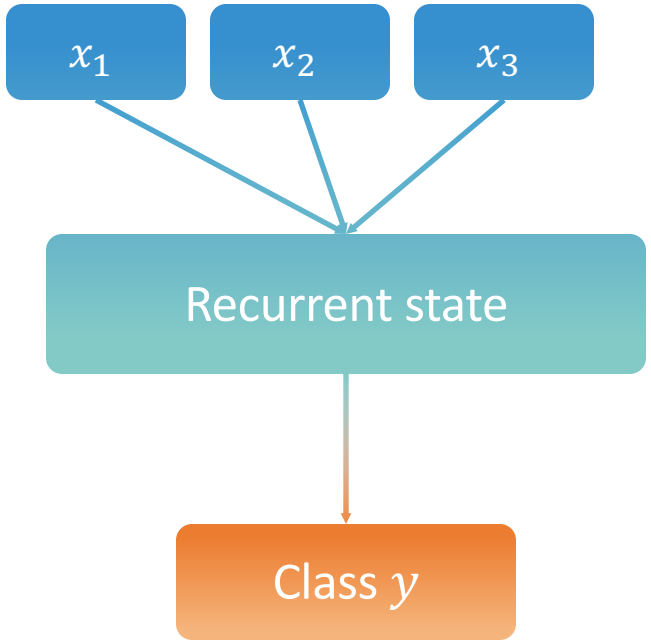
A sequence of convolutional layers, interspersed by pooling, normalization, and activation functions



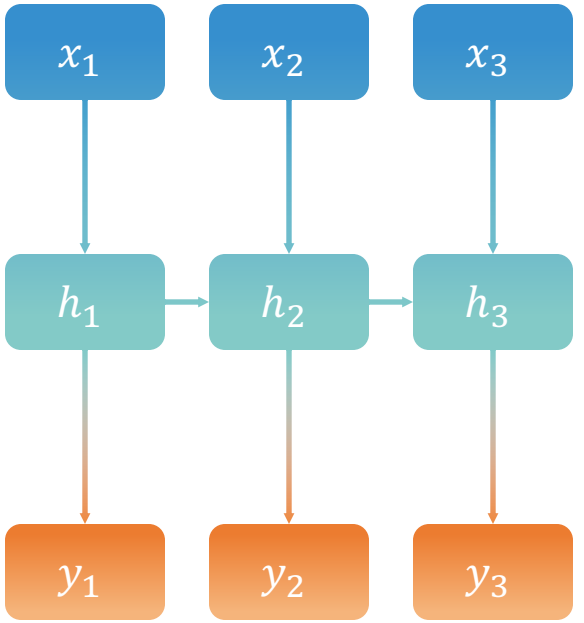
RNNs: Sequence Tasks

上海创智学院
Shanghai Innovation Institute

Sequence classification

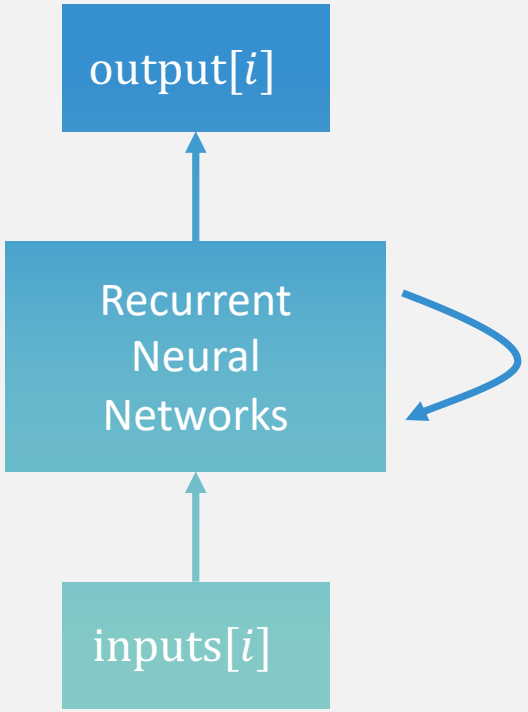


Output at each step



An internal state connects successive time steps.

Recurrent Neural Networks

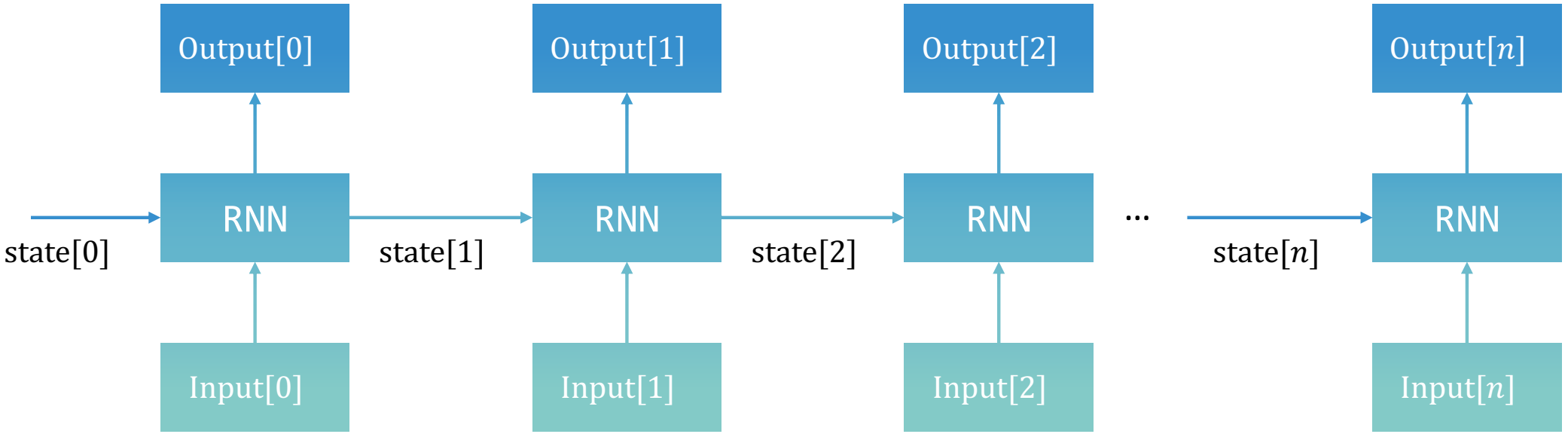


Arbitrary number of outputs

Key idea: RNNs have an internal state that is updated as a sequence is processed

Arbitrary number of inputs

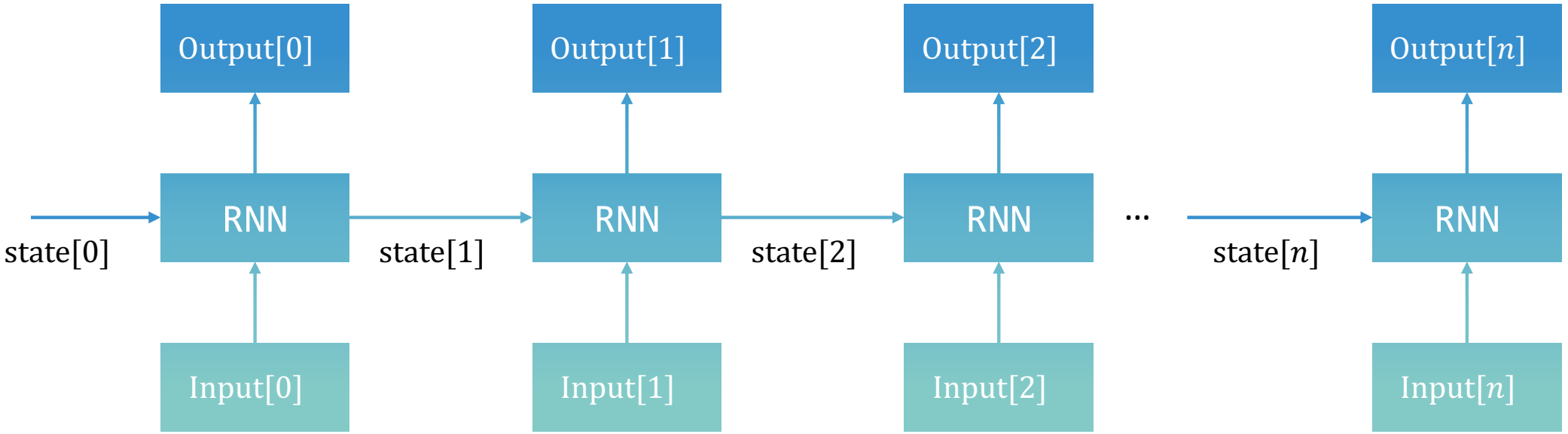
Representing RNNs in Computation Graphs



Inefficiency in RNNs?

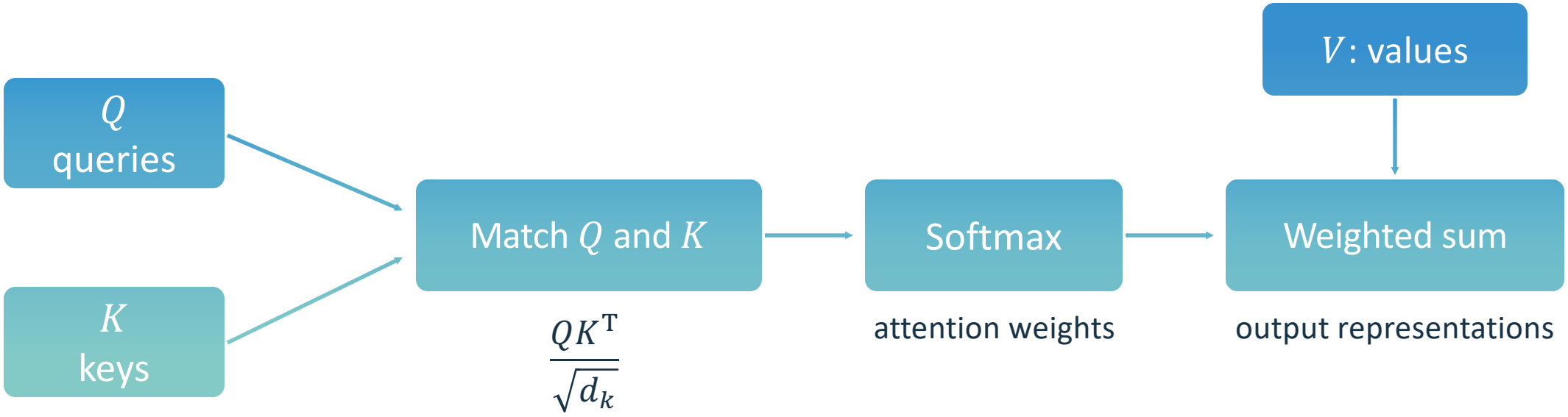
上海创智学院
Shanghai Innovation Institute

- Time-step dependencies limit parallelism within a sequence.
- A standard RNN has an $O(T)$ chain of dependent steps.
- Matrix operations within a step and different samples can still run in parallel.



Attention: Parallelism across Positions

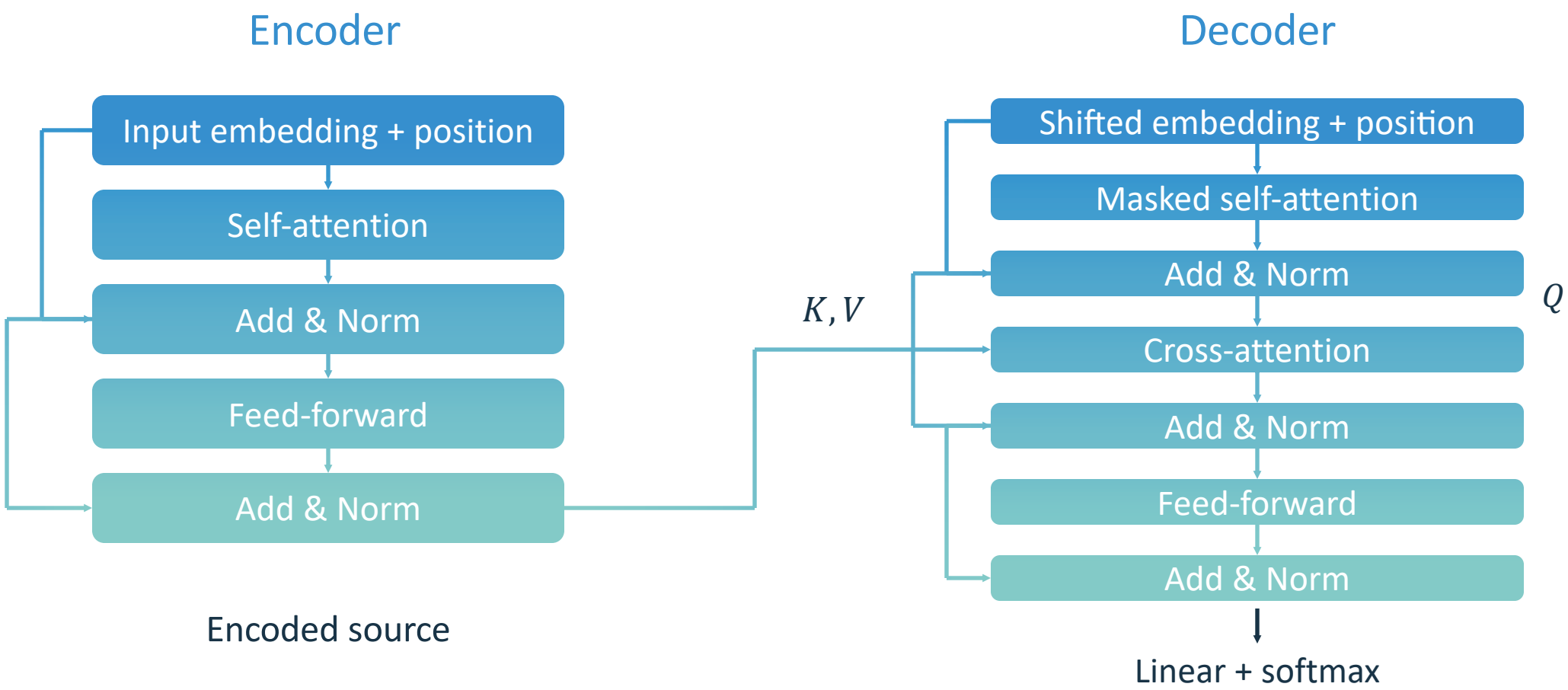
Queries compare with keys; the resulting weights combine values.



Training / prefill: known positions can be processed in parallel

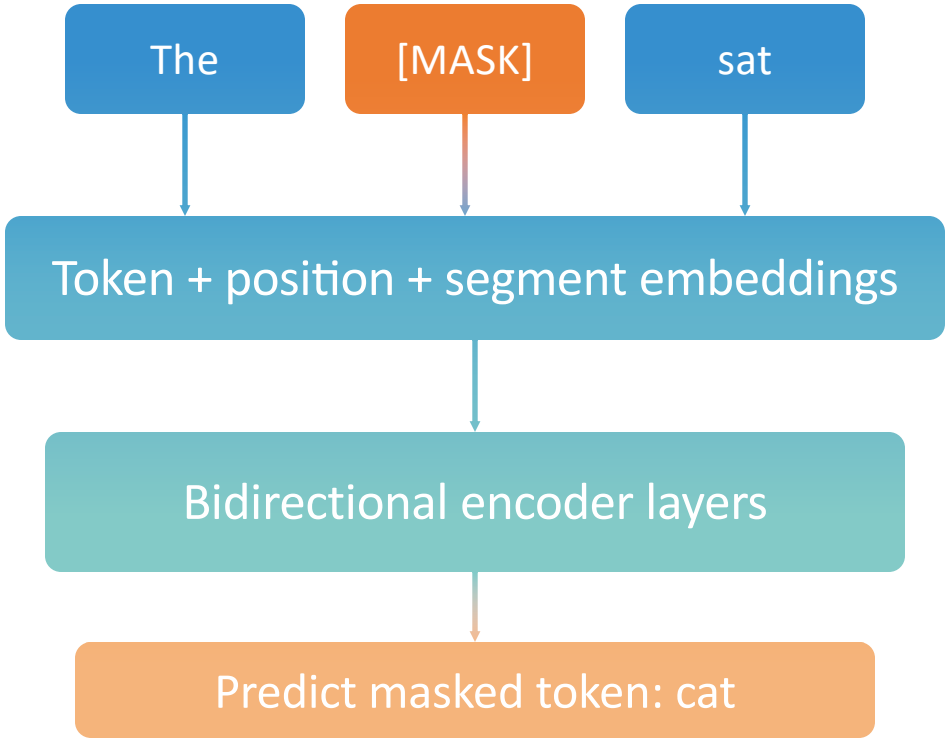
Autoregressive generation: new tokens are generated step by step

The Original Transformer (2017)



Encoder reads the source. Decoder predicts the target sequence.

BERT: Bidirectional Encoder (2018)



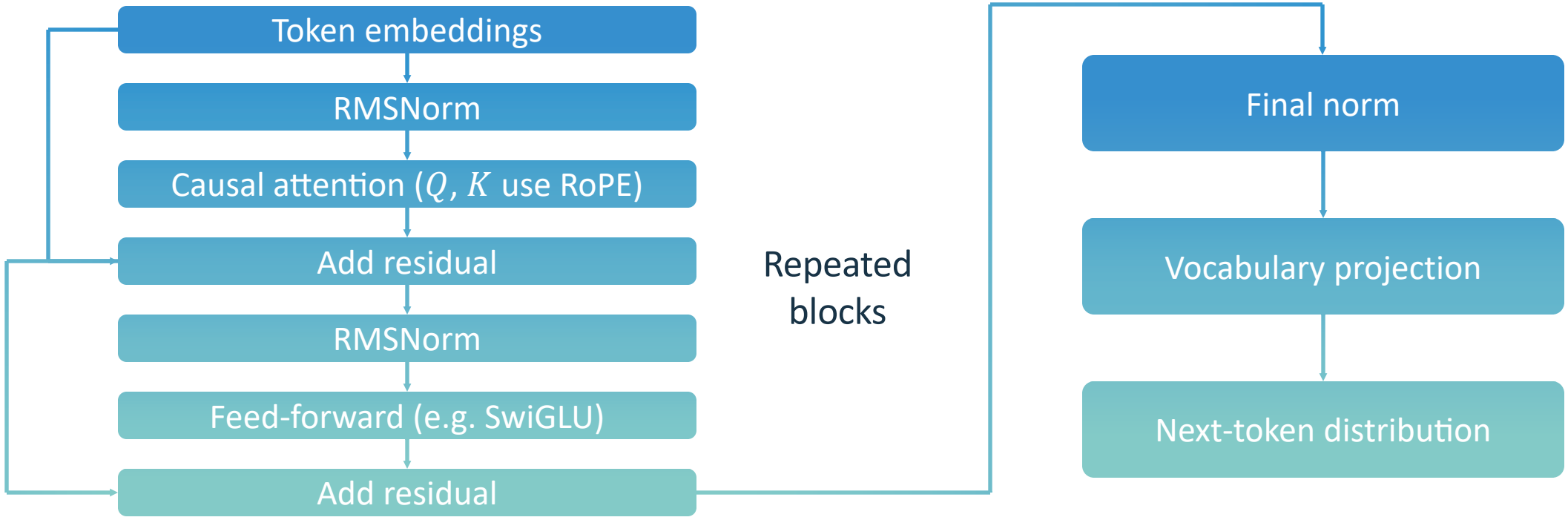
Visible attention positions



Every position can read both sides

Encoder-only architecture. Masked-token prediction uses bidirectional context.

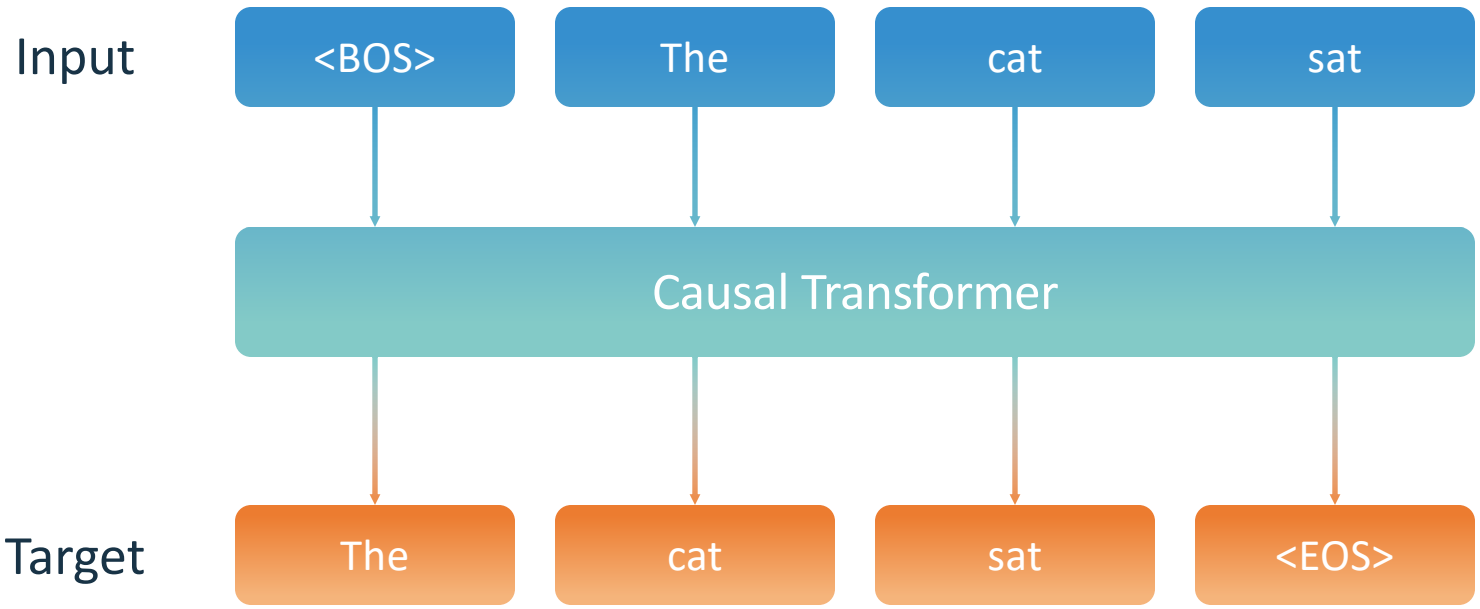
Decoder-only Autoregressive Transformer



$$p(x_{t+1} \mid x_1, \ldots, x_t)$$

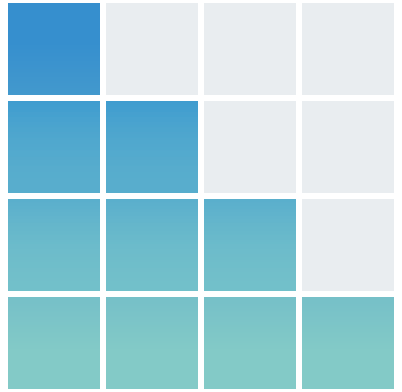
A causal mask limits each position to its available prefix.

Autoregressive Training



Cross-entropy against the shifted targets

Causal mask

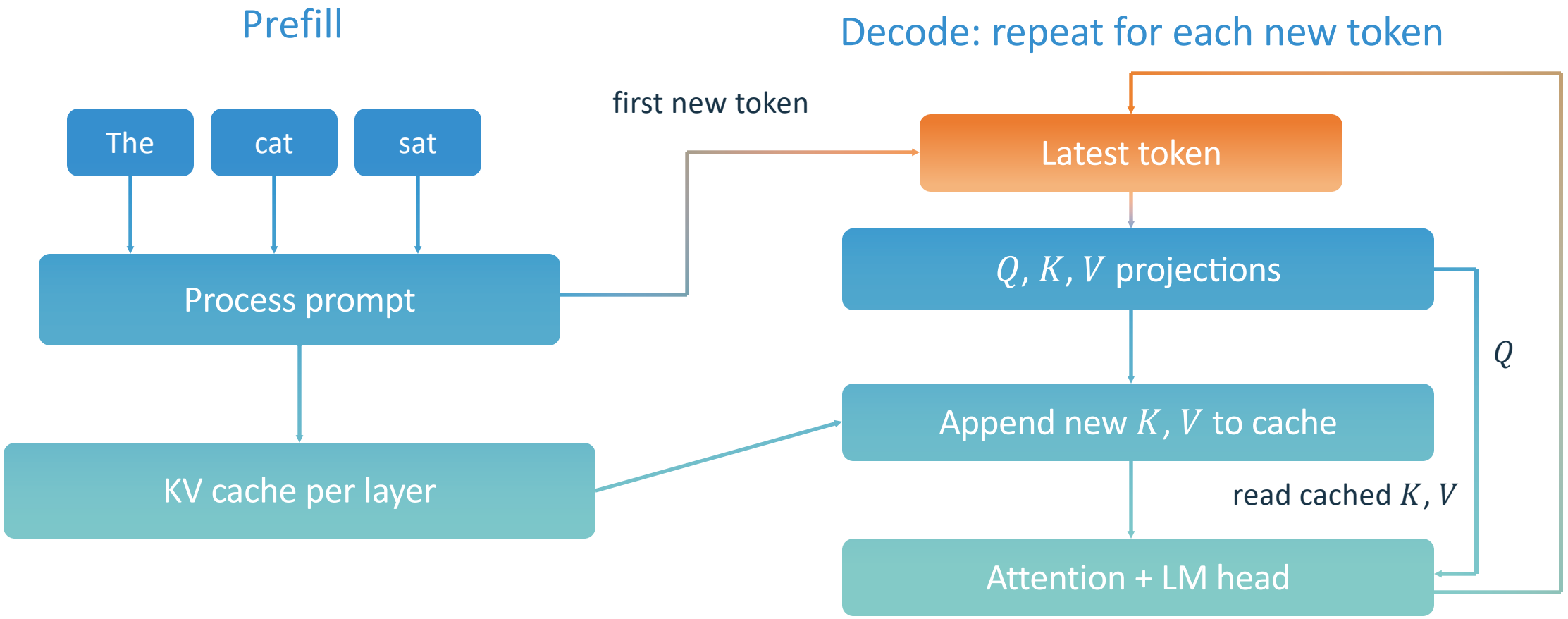


rows: queries / columns: keys

Blue = visible
Gray = future

All training targets are known. Causality still holds at every position.

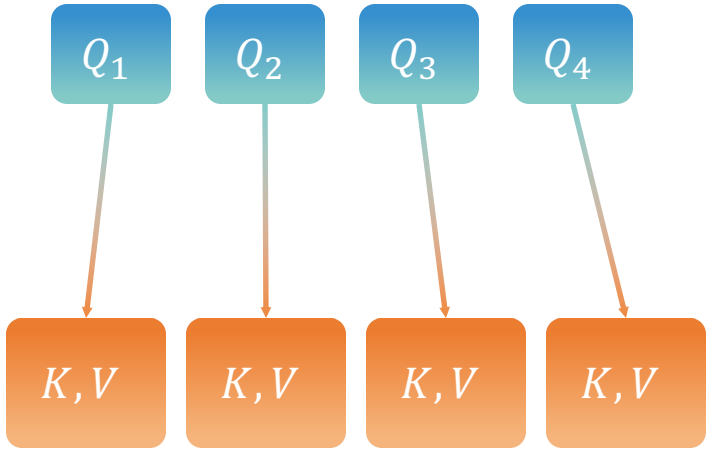
Prefill, Decode, and KV Cache



Weights stay fixed. The request's KV cache grows as generation continues.

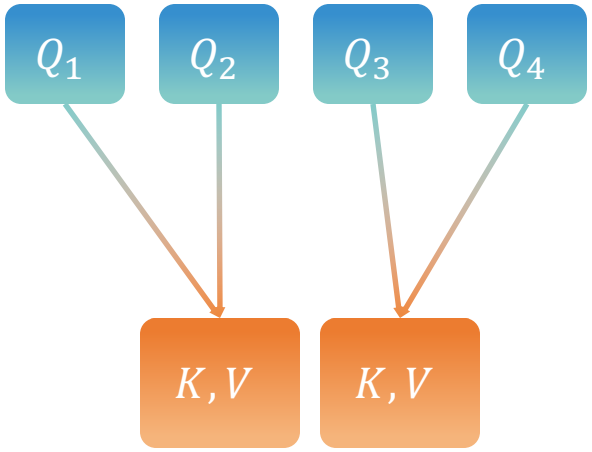
MHA, GQA, and MQA

MHA



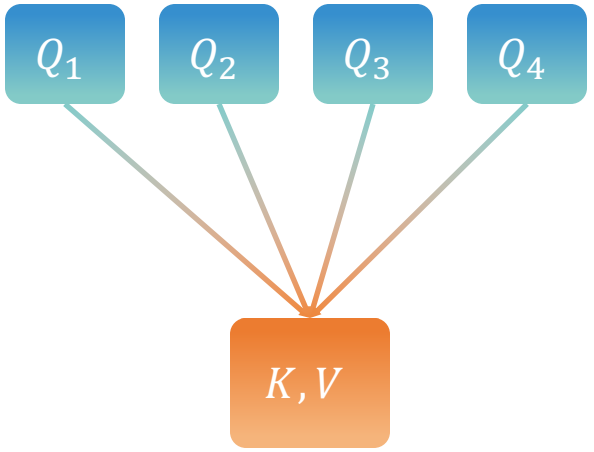
4 KV heads

GQA



2 KV heads

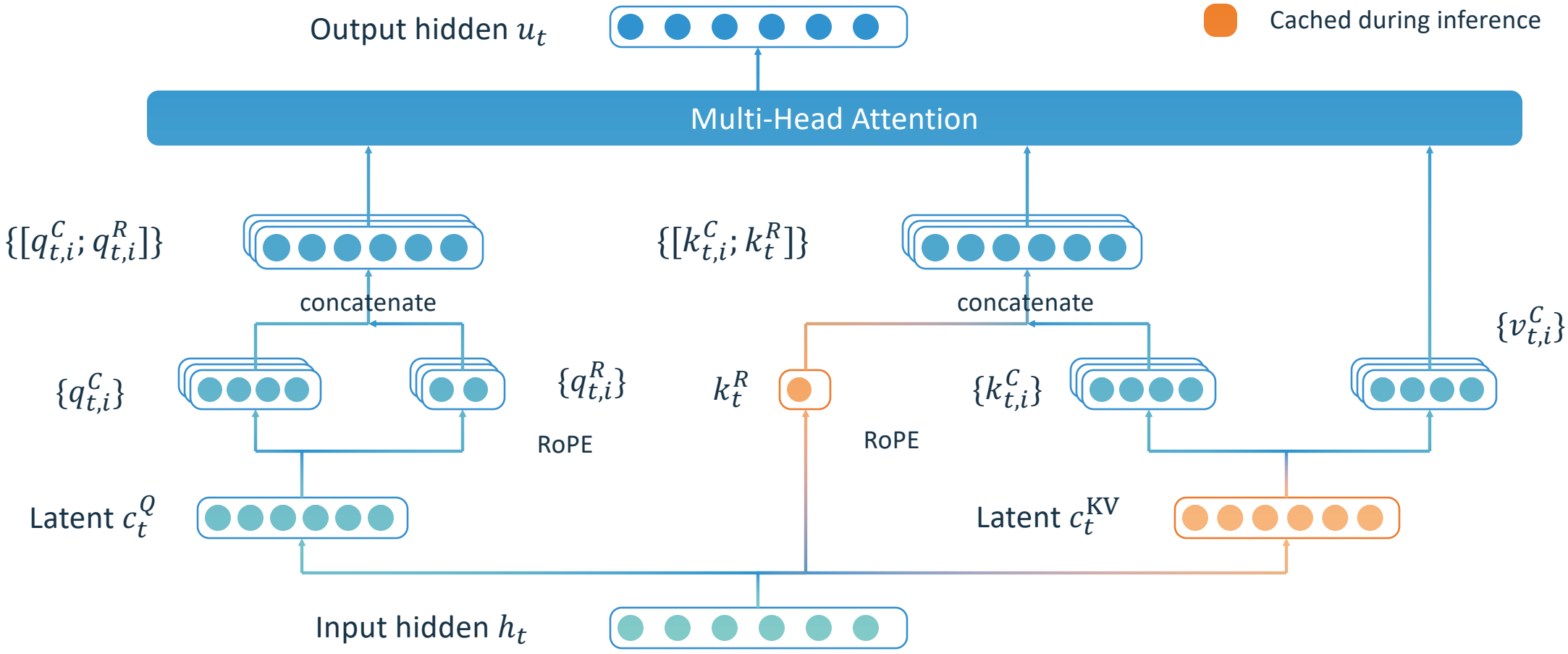
MQA



1 KV head

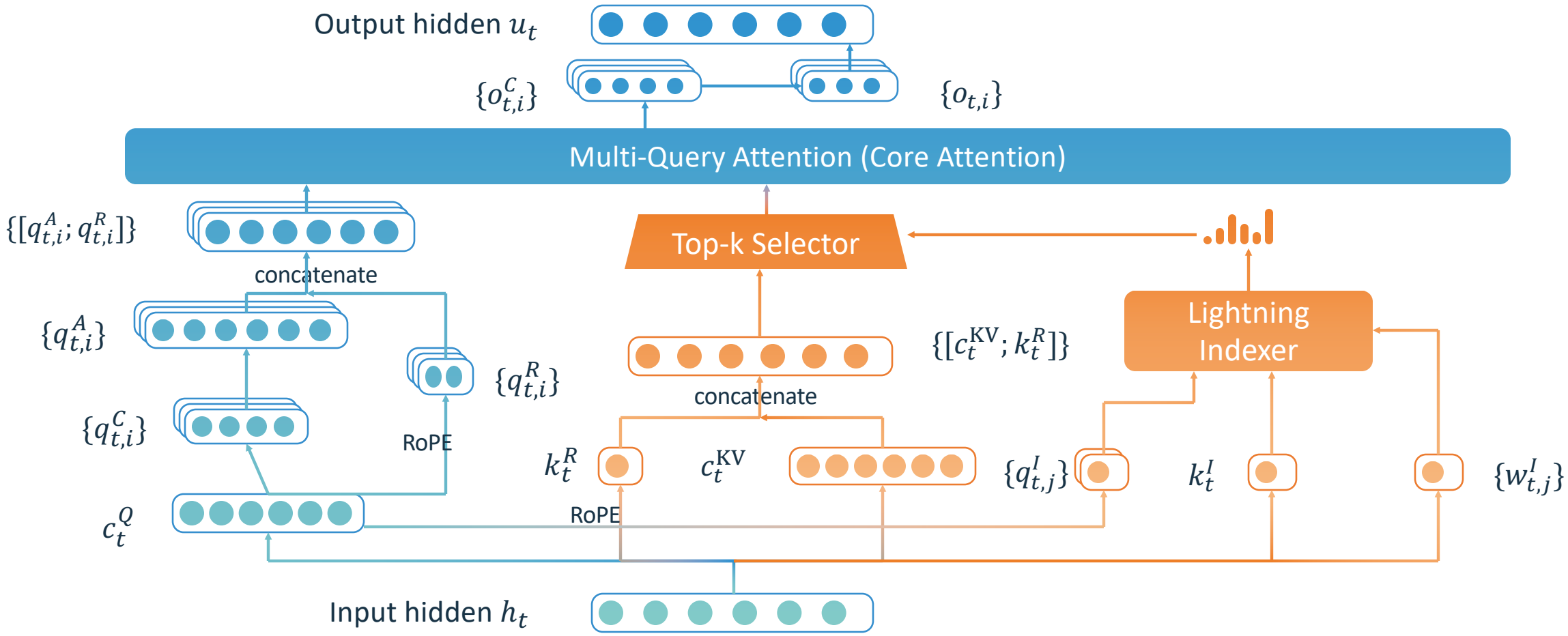
Fewer KV heads reduce stored values per token.

Multi-head Latent Attention (MLA)



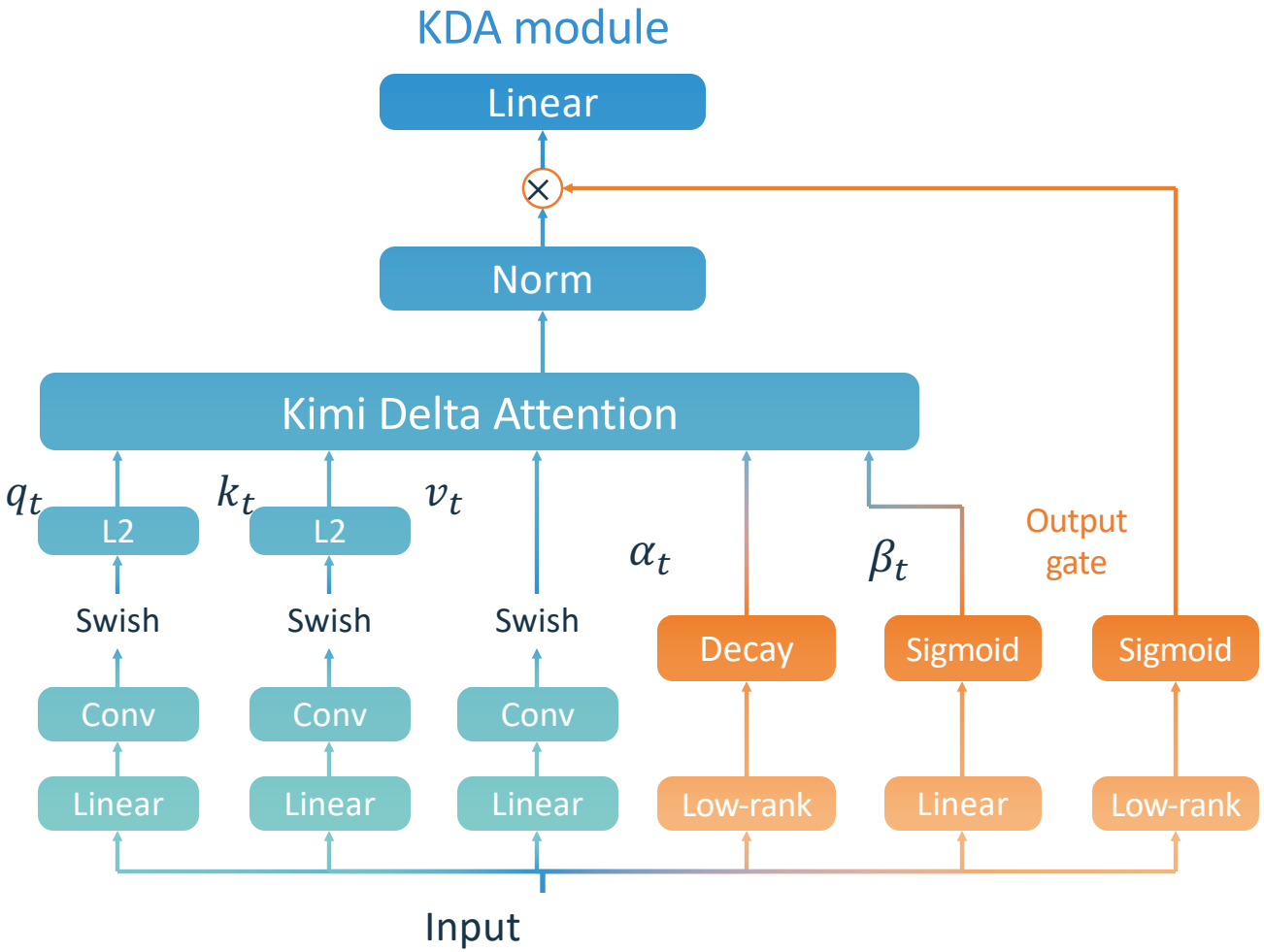
MLA compresses features per token. History still contains many tokens.

DeepSeek Sparse Attention (DSA)



The indexer chooses entries. Core attention reads only the selected KV.

Kimi Delta Attention (KDA)



Recurrent state update

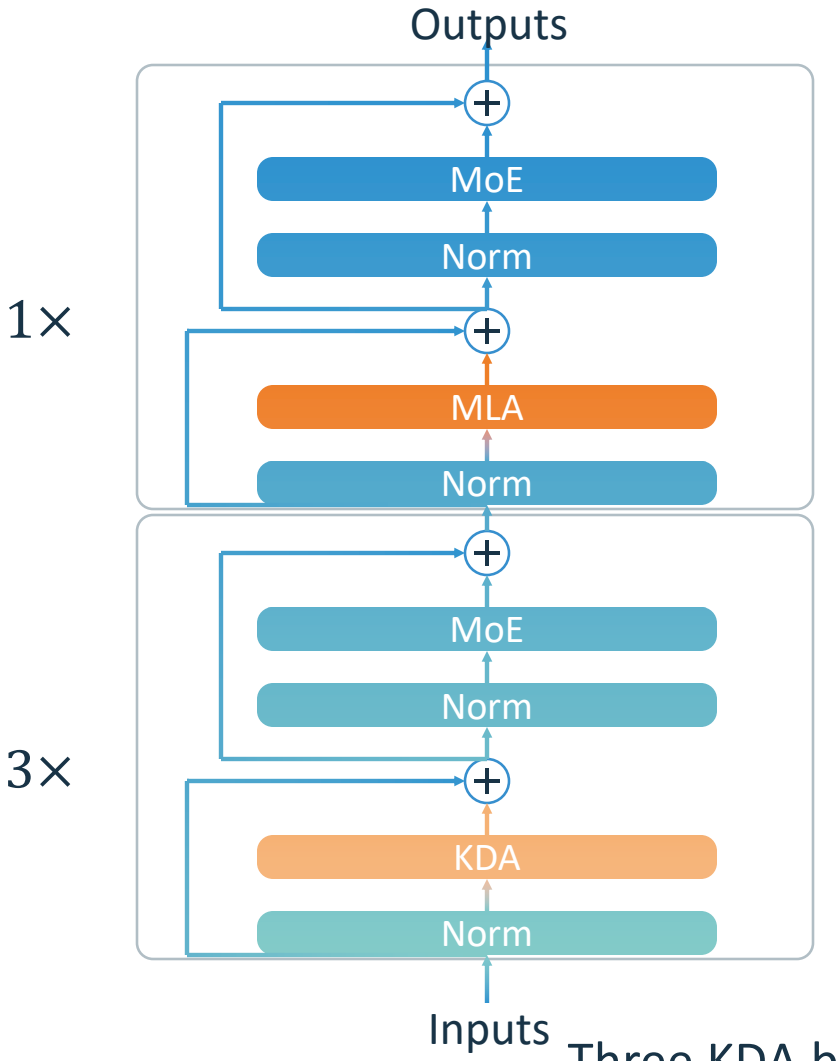
$$\bar{S}_t = \mathrm{Diag}(\alpha_t) S_{t-1}$$

$$e_t = v_t - \bar{S}_t^{\mathrm{T}} k_t$$

$$S_t = \bar{S}_t + \beta_t k_t e_t^{\mathrm{T}}$$

$$o_t = S_t^{\mathrm{T}} q_t$$

Kimi Linear: KDA and MLA Layers



KDA training uses chunks

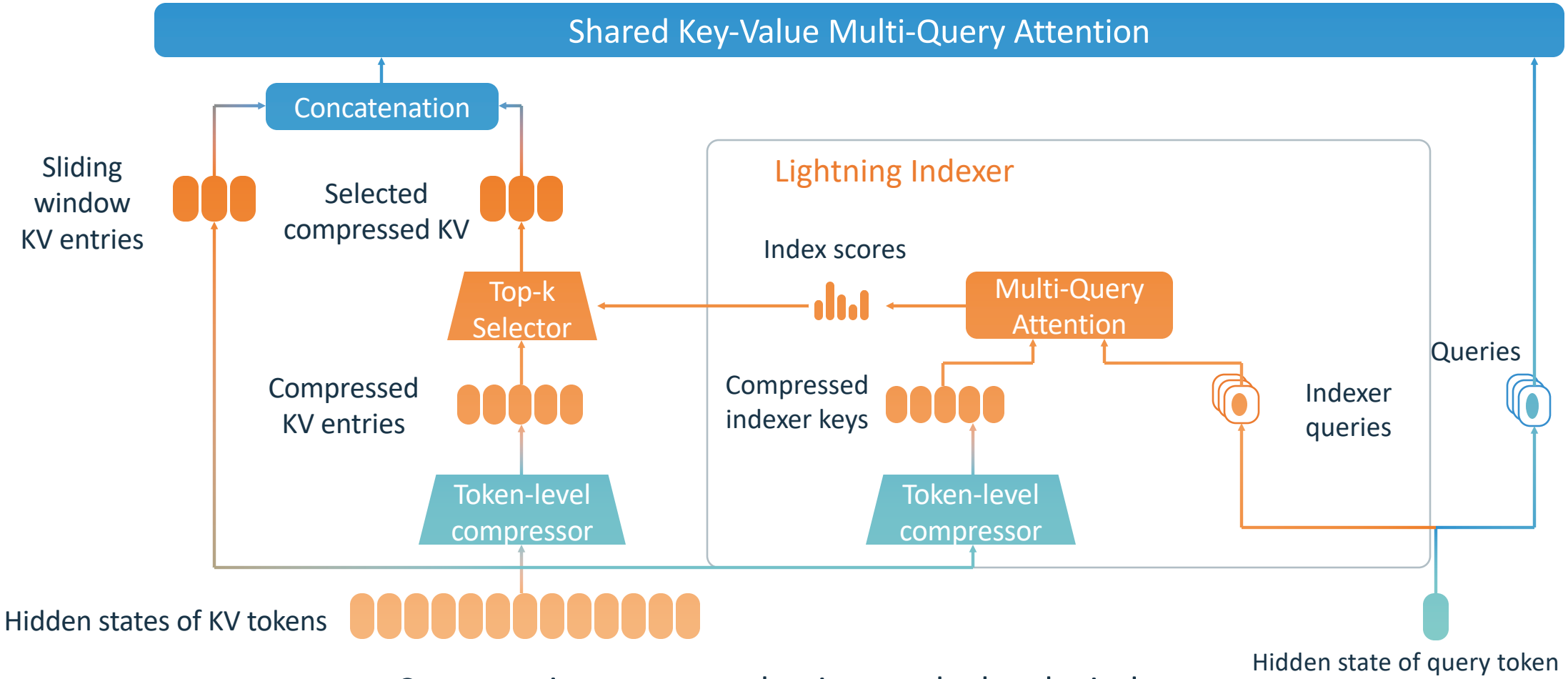


Parallel work inside each chunk

State passes between chunks

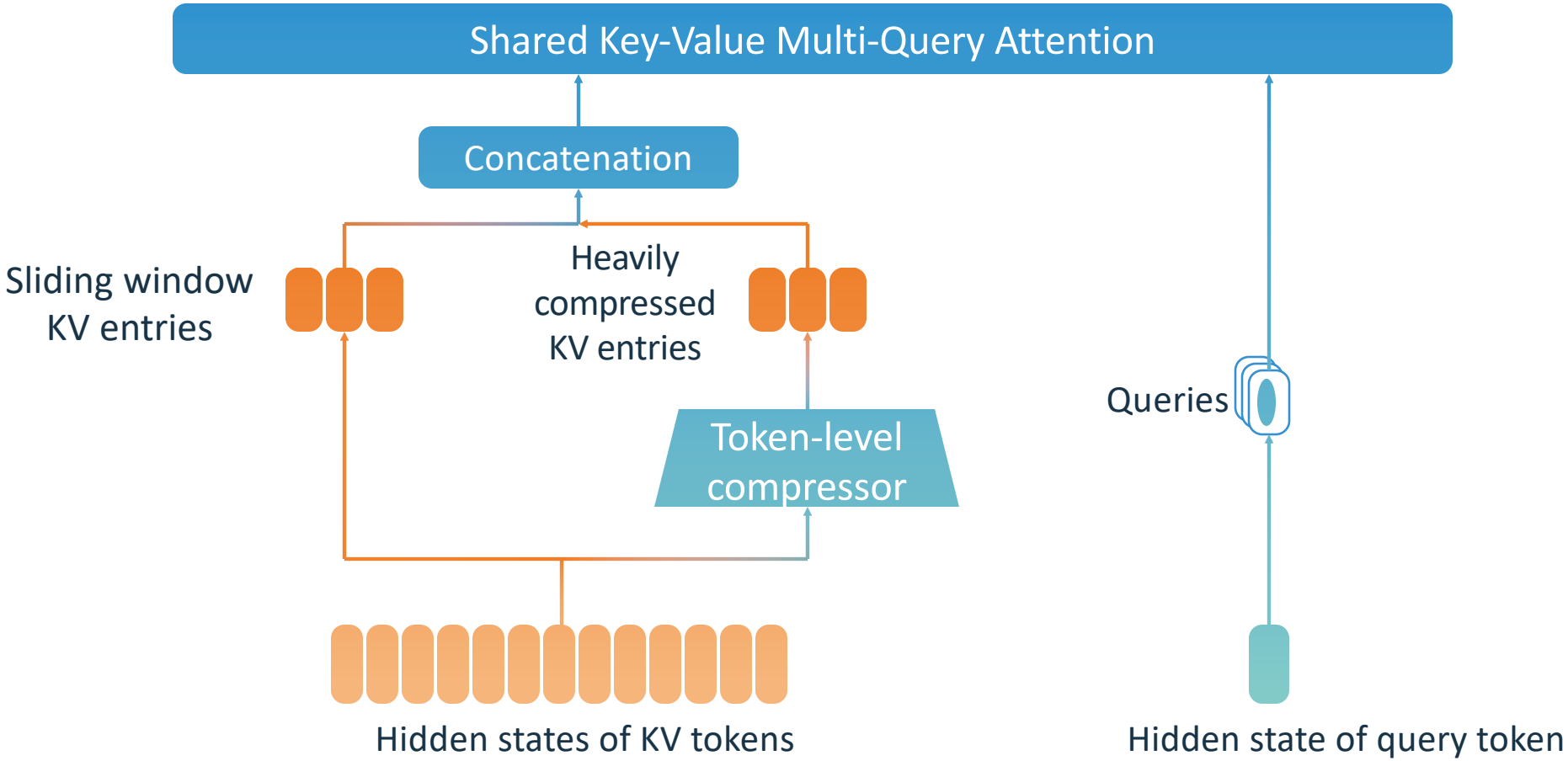
Three KDA blocks followed by one MLA block

CSA: Compressed Sparse Attention



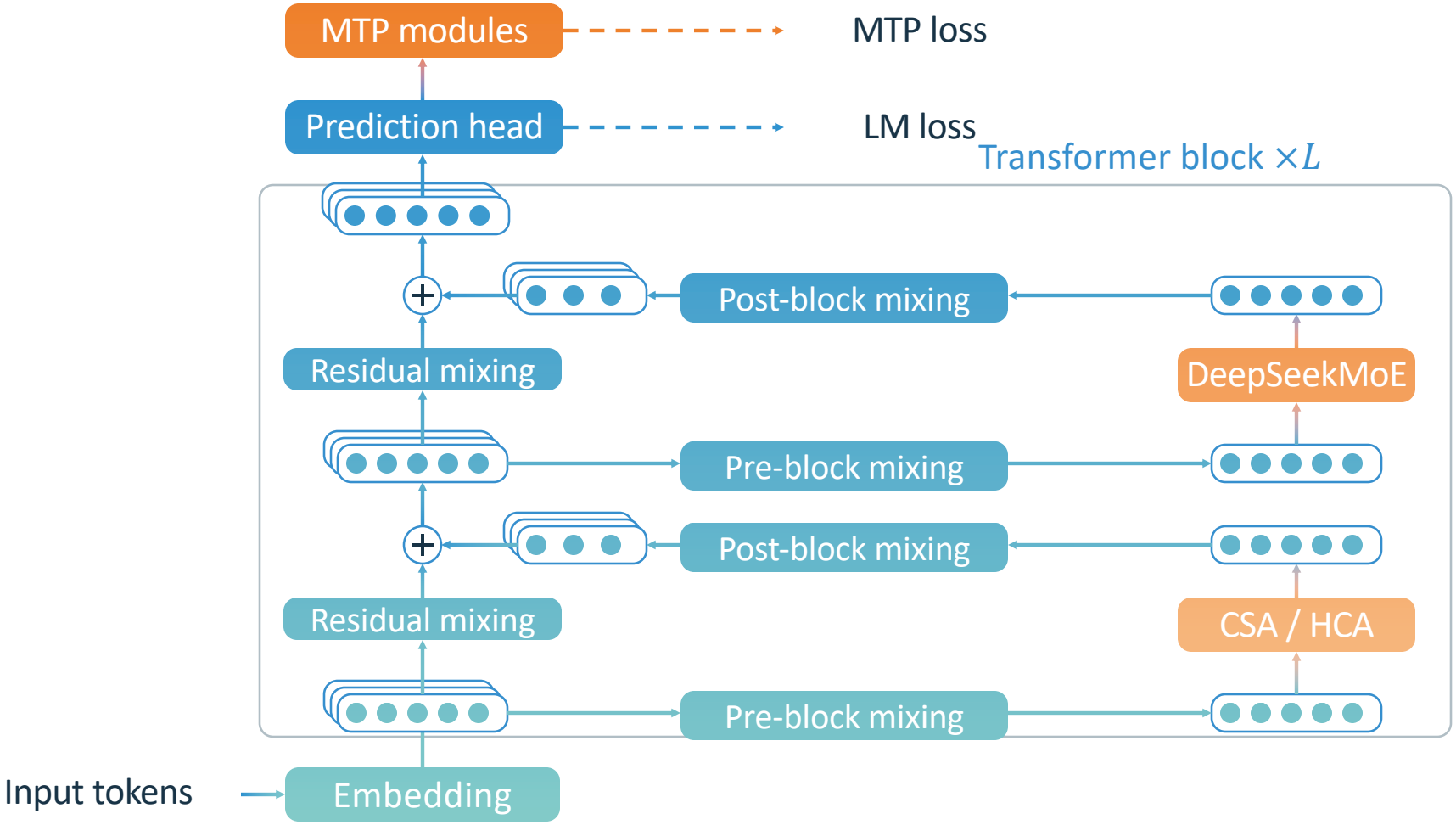
Compression, sparse selection, and a local window

HCA: Heavily Compressed Attention



Heavy compression and a local window, without a top-k indexer

DeepSeek-V4: Hybrid Attention



CSA / HCA provide attention. mHC mixes residual streams around each sublayer.

Attention Variants: What Changes?

上海创智学院
Shanghai Innovation Institute

Mechanism	Stored history	Core attention / state read
MLA	Latent vector per token	All historical latent entries
DSA	Latent entries + index keys	Top-k entries, selected by indexer
KDA	Fixed-size matrix state	State read with current query
CSA	Compressed blocks + local KV	Top-k compressed entries + local KV
HCA	Heavily compressed blocks + local KV	All compressed entries + local KV

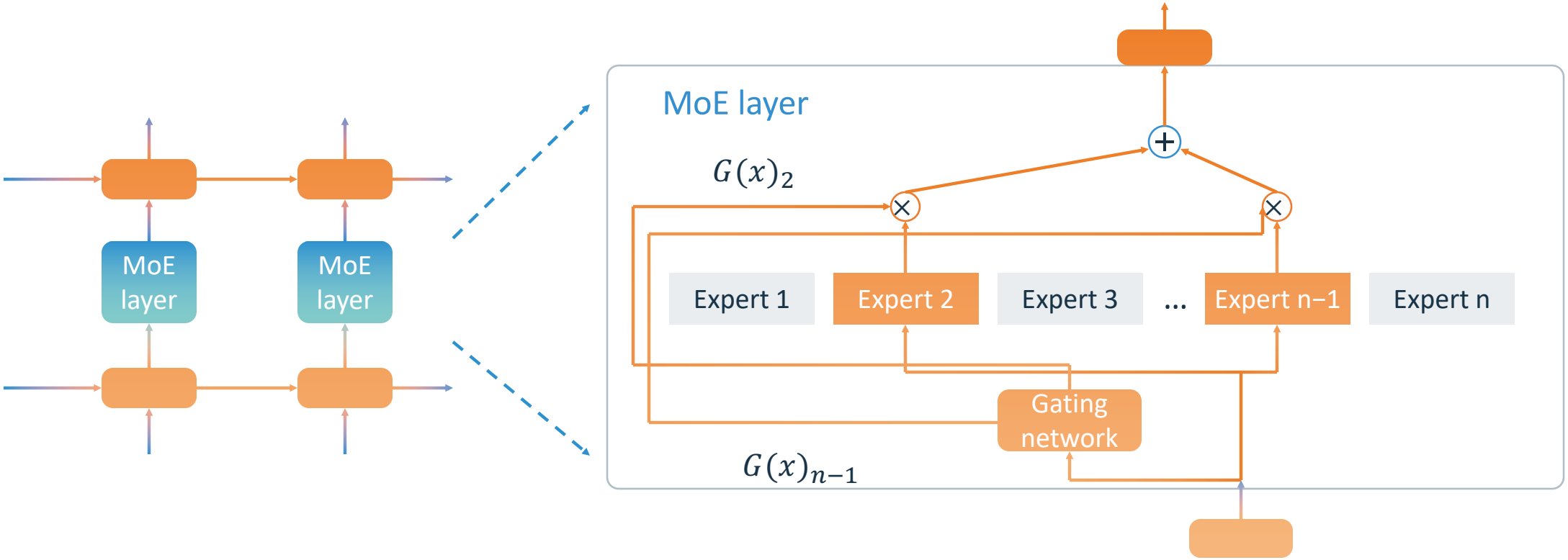
Which method reduces stored history, and which reduces the entries read?

Mixture-of-Experts

上海创智学院
Shanghai Innovation Institute

A router selects a small set of experts for each token.

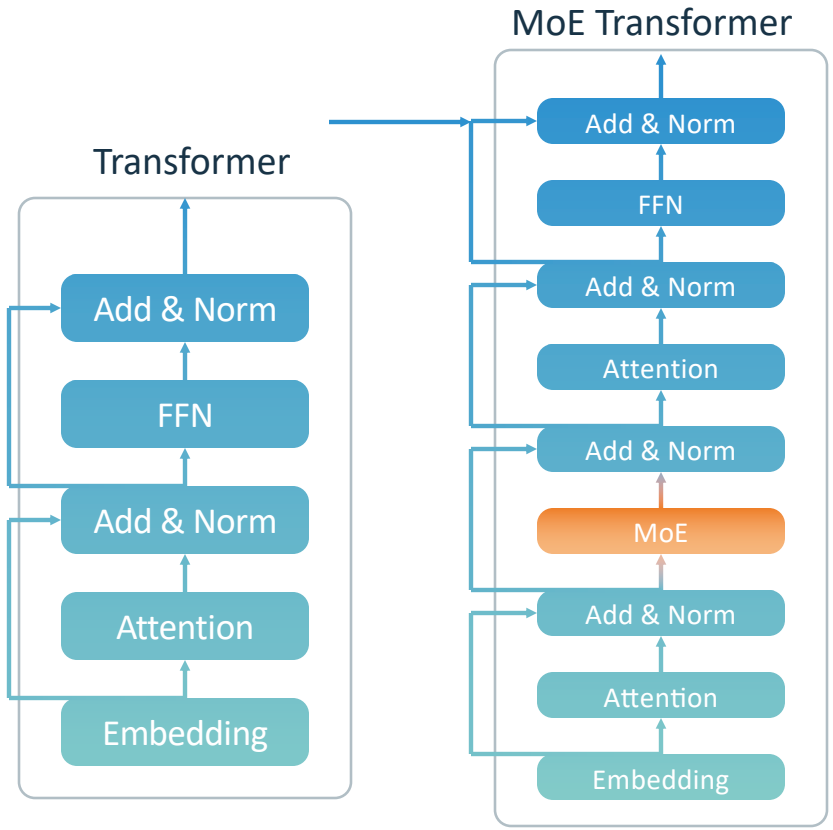
Sparse activation separates total parameters from active parameters.



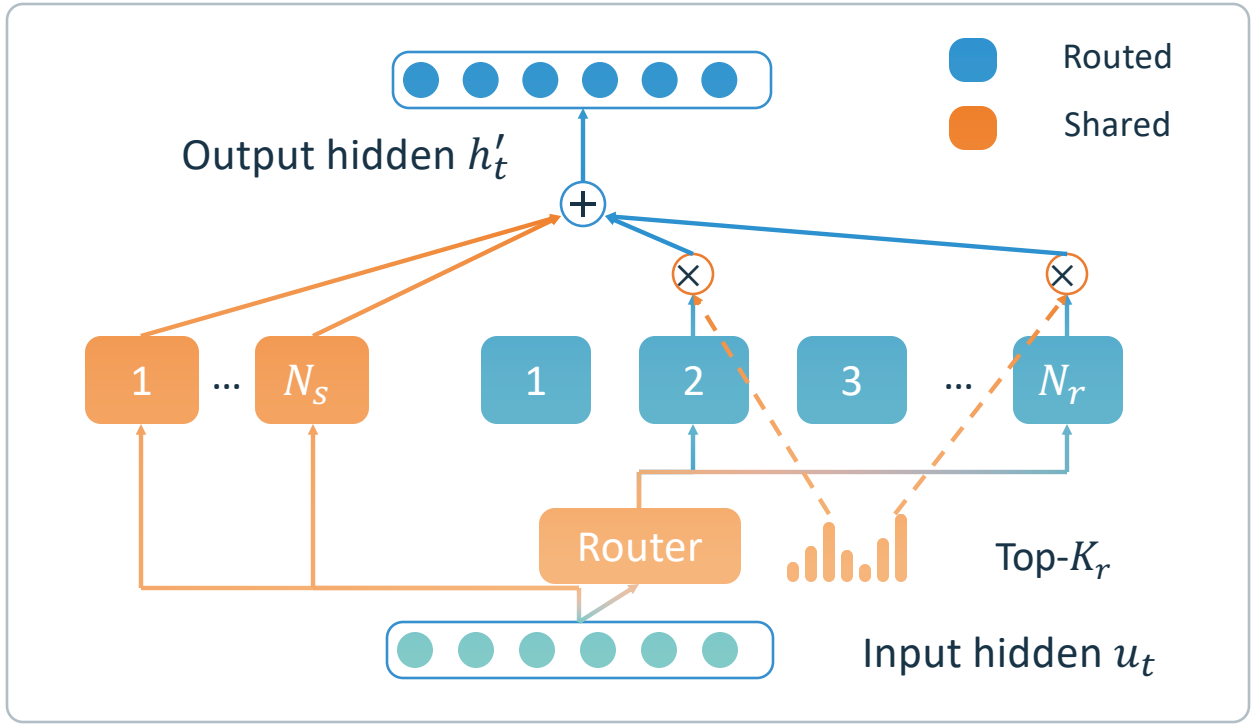
MoE Large Language Models

上海创智学院
Shanghai Innovation Institute

MoE inside a Transformer



DeepSeek-V3: routed and shared experts



Only selected routed experts process each token.

02

Overview of ML Systems

An Overview of Machine Learning Systems

上海创智学院
Shanghai Innovation Institute



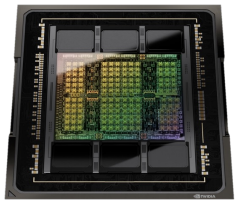
ML Models

Automatic Differentiation

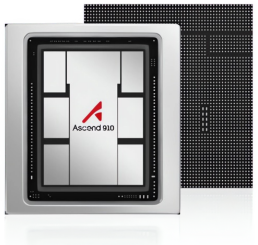
Graph Optimization

Parallel and Distributed Execution

Hardware Acceleration



NVIDIA GPU

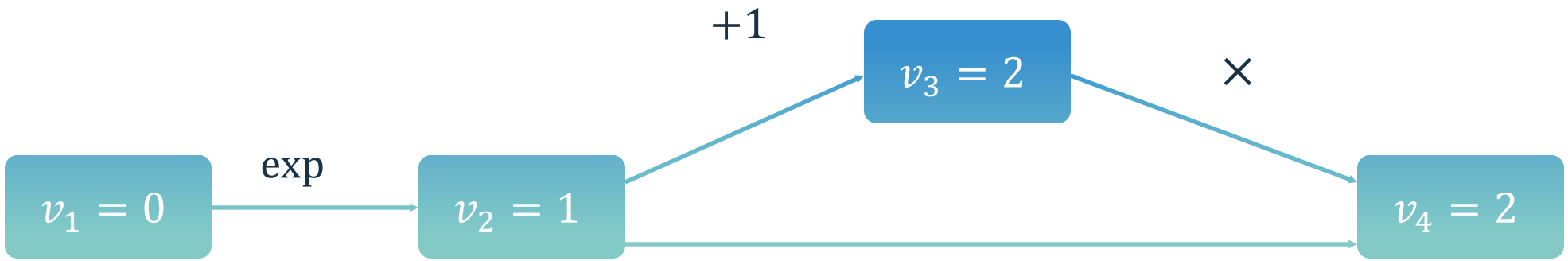


HUAWEI NPU



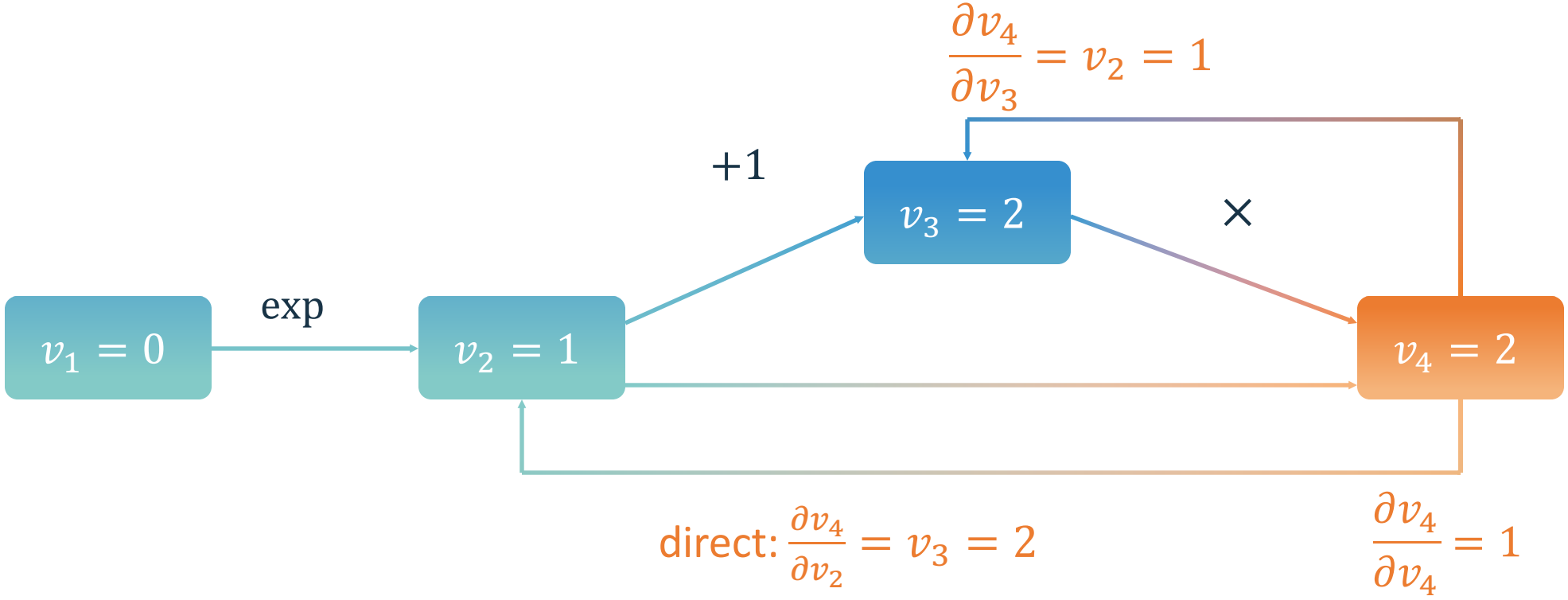
Mobile devices

Automatic Differentiation: Forward Values



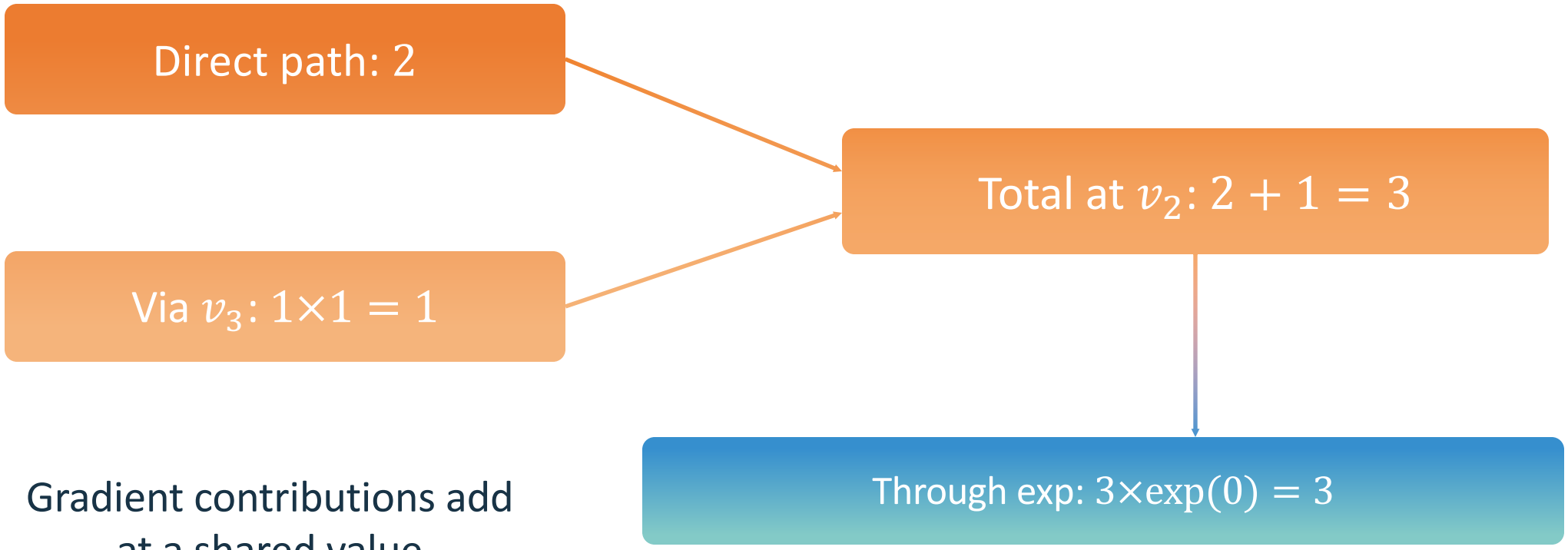
Forward execution records the values needed by the backward pass.

Backward Step 1: Local Derivatives



The multiplication sends one gradient contribution to each input.

Backward Step 2: Accumulate Paths



Gradient contributions add at a shared value

Backward: local chain rules + accumulation over every downstream path.

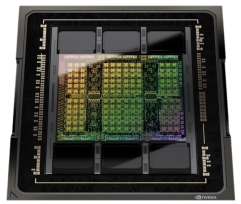
Layer 1: Automatic Differentiation

上海创智学院
Shanghai Innovation Institute



ML Models

- Automatic Differentiation
- Graph Optimization
- Parallel and Distributed Execution
- Hardware Acceleration



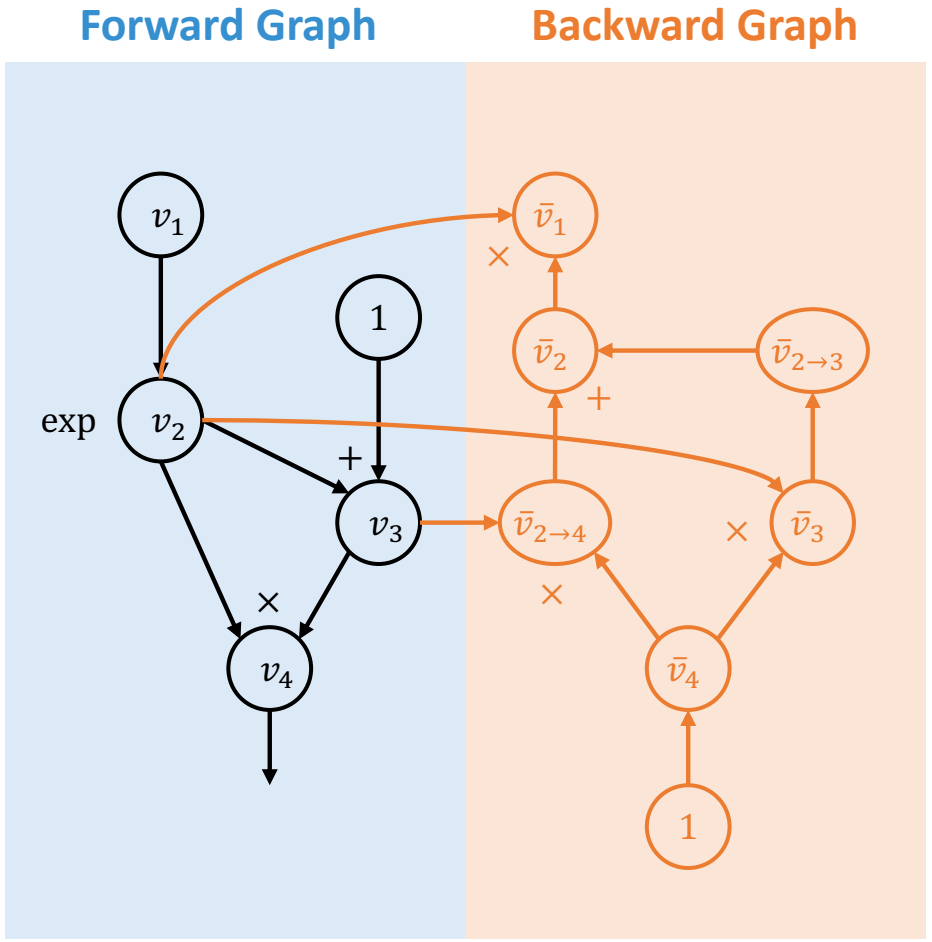
NVIDIA GPU



HUAWEI NPU



Mobile devices

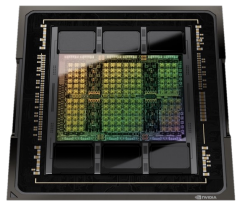


Layer 2: Graph Optimization

上海创智学院
Shanghai Innovation Institute

ML Models

- Automatic Differentiation
- Graph Optimization
- Parallel and Distributed Execution
- Hardware Acceleration



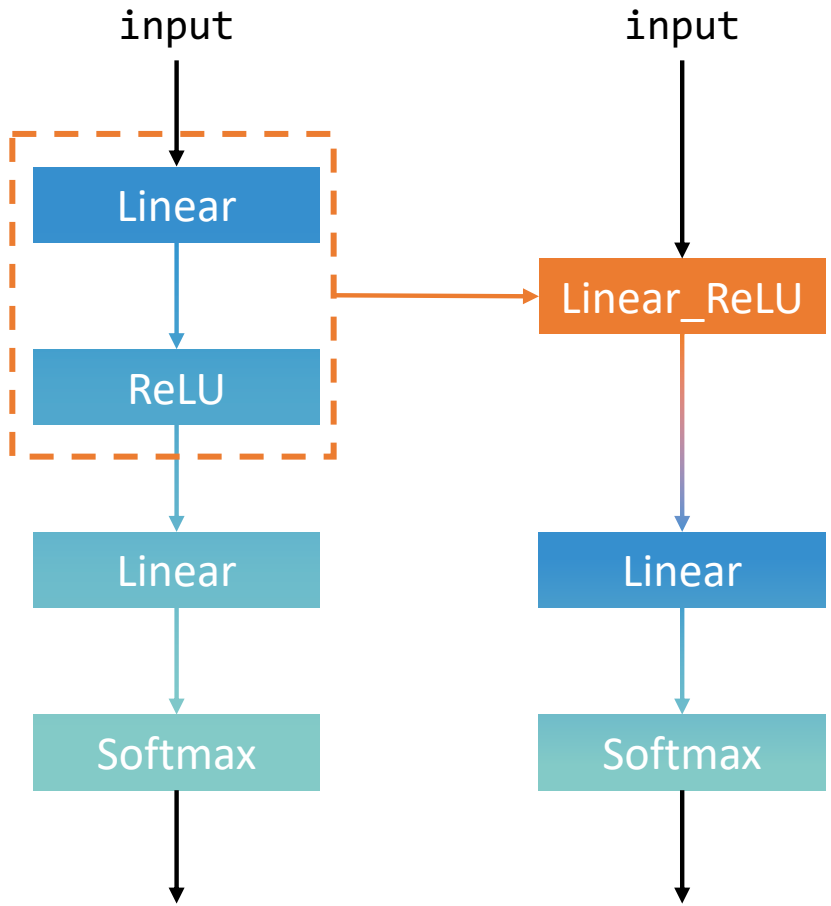
NVIDIA GPU



HUAWEI NPU

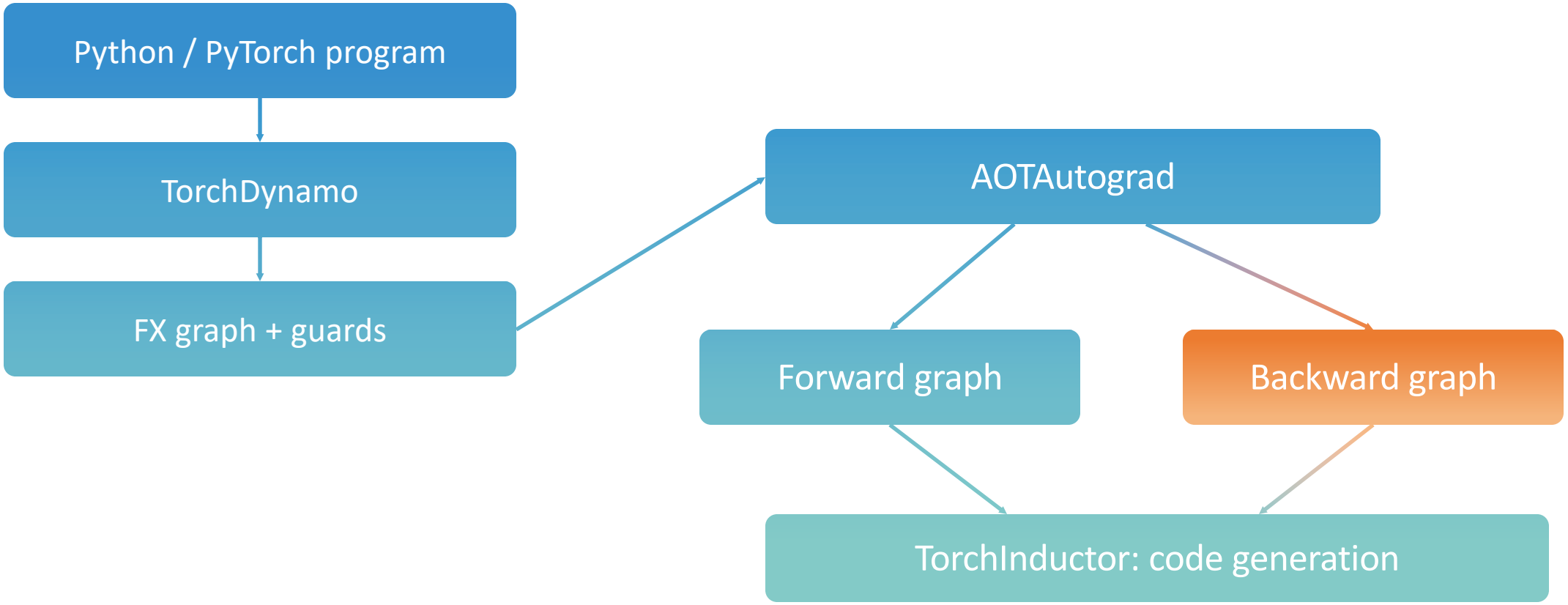


Mobile devices



Why can kernel fusion improve performance?

torch.compile: Compilation Pipeline

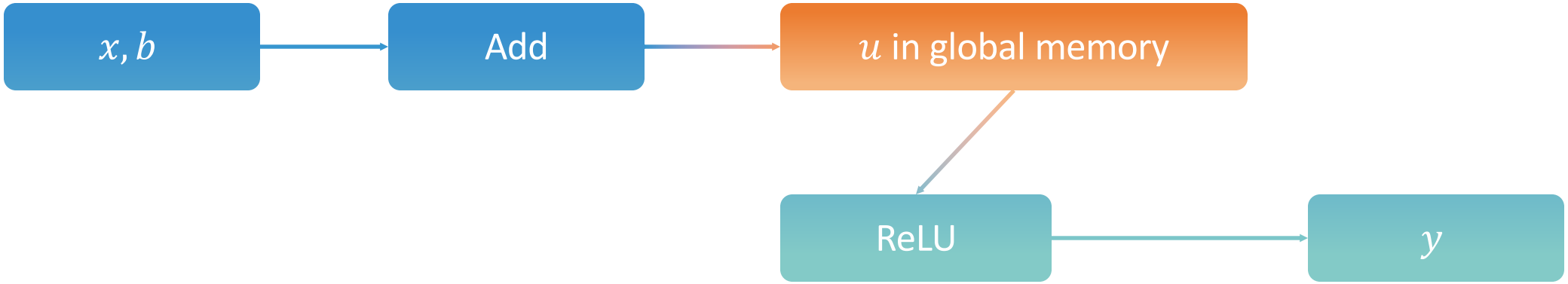


Capture graphs, construct gradients, then lower operations to executable kernels.

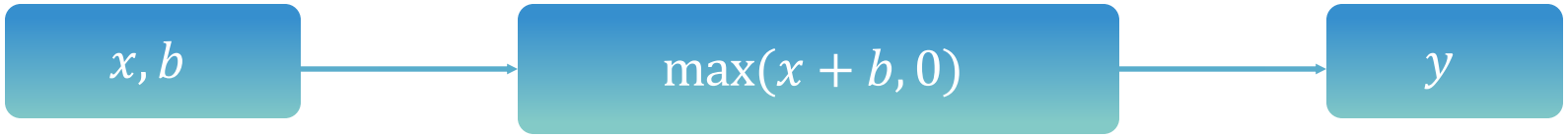
Kernel Fusion: A Memory-Traffic Example

上海创智学院
Shanghai Innovation Institute

Separate kernels



Fused kernel

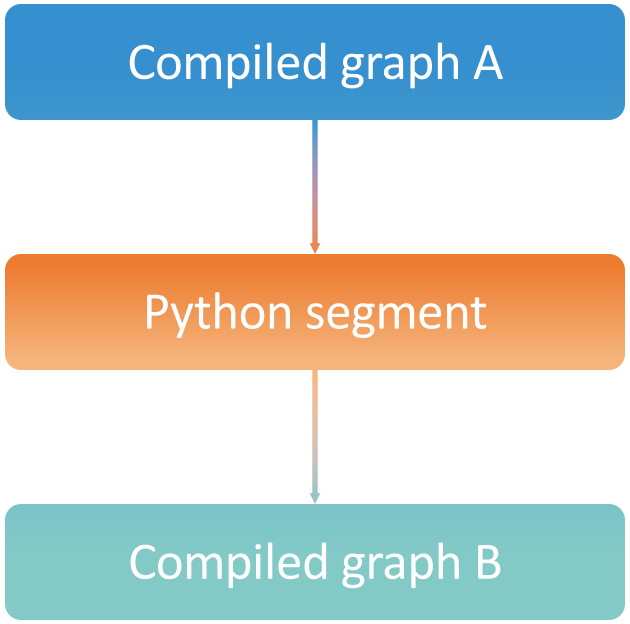


Fusion can avoid writing and rereading an intermediate tensor.

Graph Breaks and Guards

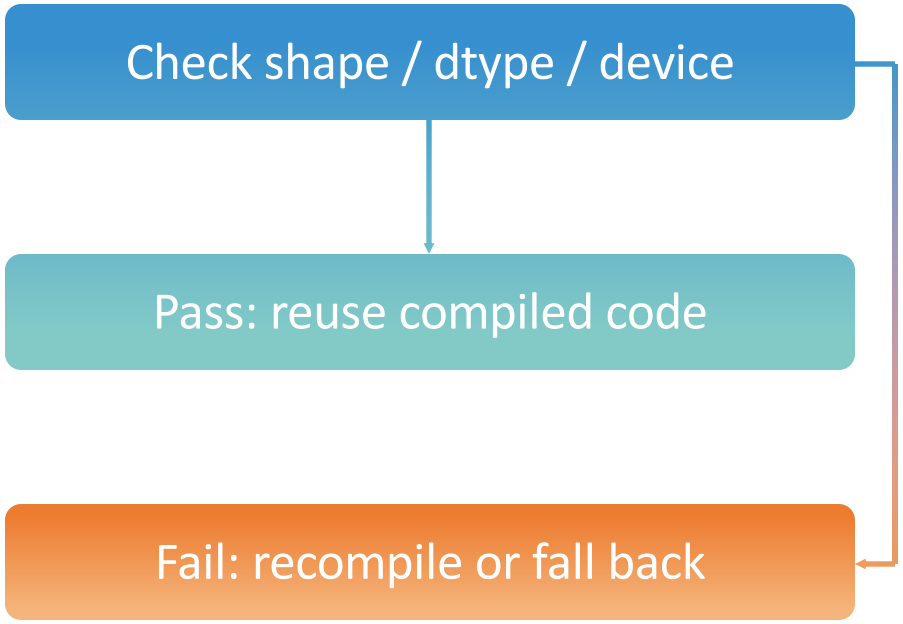
上海创智学院
Shanghai Innovation Institute

Graph break: capture is split



e.g. data-dependent Python control flow

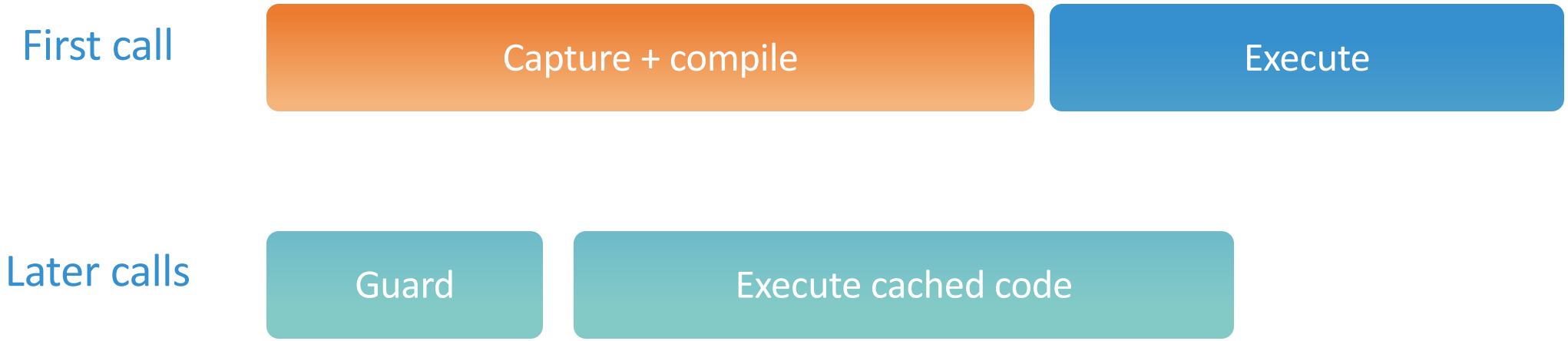
Guard: reuse conditions



Graph breaks split captured regions. Guard failures can trigger recompilation.

Measuring torch.compile

上海创智学院
Shanghai Innovation Institute



same applicable inputs

$$\text{Break-even calls} \approx \frac{\text{extra compile time}}{\text{time saved per call}}$$

Check numerical agreement, warm up, then time representative repeated execution.

Layer 3: Parallelization

上海创智学院
Shanghai Innovation Institute

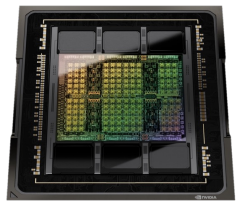
ML Models

Automatic Differentiation

Graph Optimization

Parallel and Distributed Execution

Hardware Acceleration



NVIDIA GPU



HUAWEI NPU

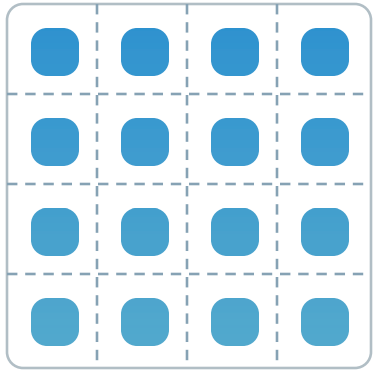


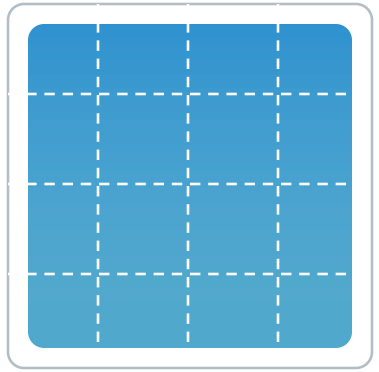
Mobile devices

Data parallelism

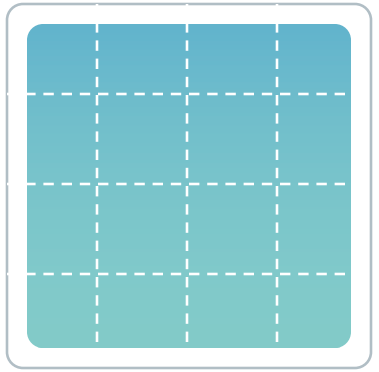
Model parallelism

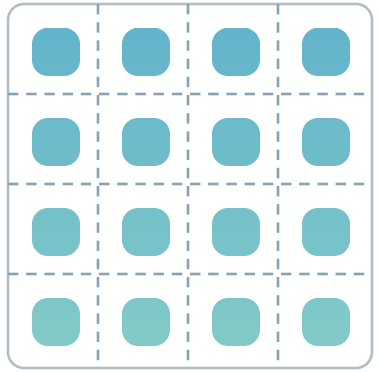
Weights





Data





Layer 4: Hardware Acceleration

上海创智学院
Shanghai Innovation Institute

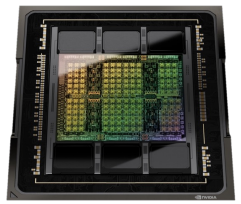
ML Models

Automatic Differentiation

Graph Optimization

Parallel and Distributed Execution

Hardware Acceleration



NVIDIA GPU

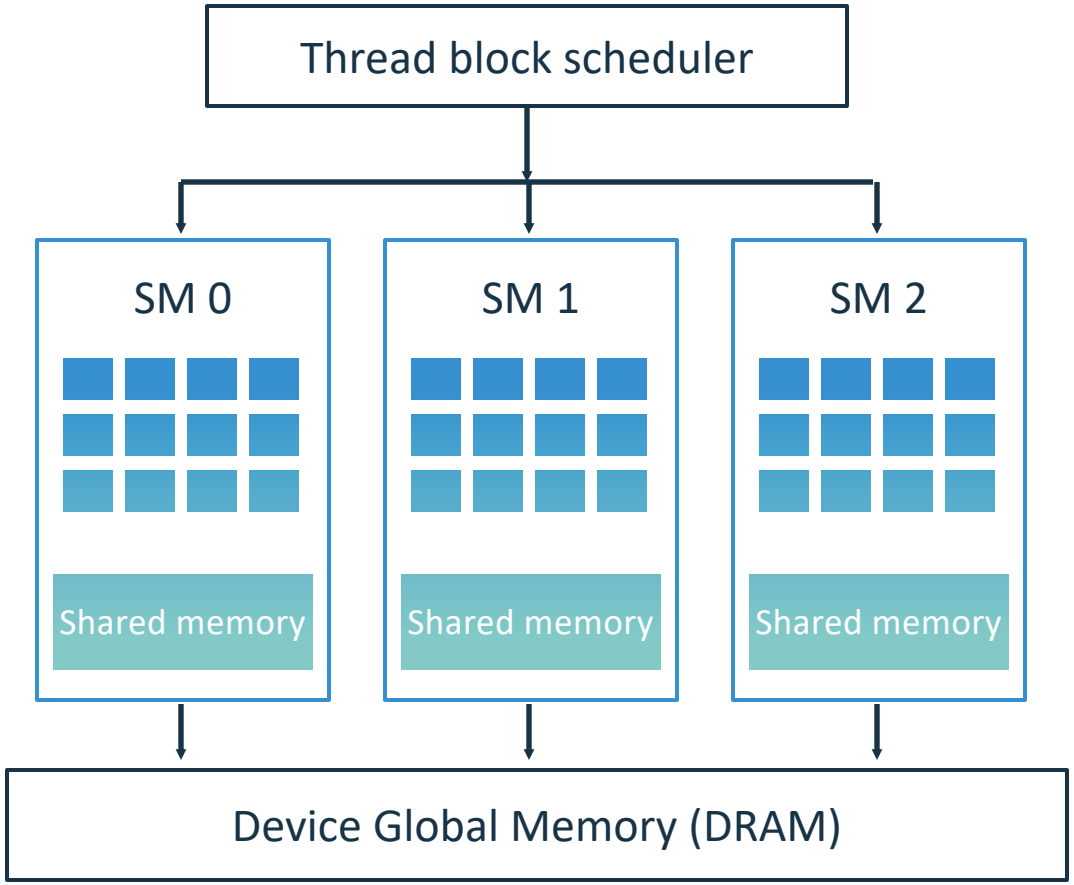


HUAWEI NPU



Mobile devices

GPU example



Cross-Layer Optimizations

上海创智学院
Shanghai Innovation Institute

ML Models

Automatic Differentiation

Graph Optimization

Parallel and Distributed Execution

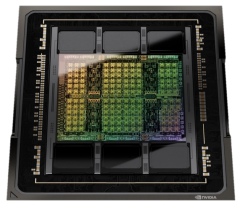
Hardware Acceleration

Training graph capture and fusion

torch.compile

Mega-Kernel

Computation–communication overlap



NVIDIA GPU



HUAWEI NPU



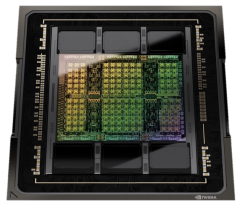
Mobile devices

Upcoming Lectures

上海创智学院
Shanghai Innovation Institute

ML Models

- Automatic Differentiation
- Graph Optimization
- Parallel and Distributed Execution
- Hardware Acceleration



NVIDIA GPU



HUAWEI NPU



Mobile devices

- Automatic Differentiation and Frameworks
- General Hardware Acceleration
- GPU Architecture and CUDA Programming
- NPU Architecture and Ascend C Programming
- Machine Learning Compilation
- LLMs and General Optimizations
- Introduction to Distributed Computing
- LLM Parallelization and Training
- LLM Serving and Inference
- LLM Post-Training and Reinforcement Learning

Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

Thanks

Siyuan Feng
Shanghai Innovation Institute